

UART



Thực hiện : Nguyễn Tuấn Anh

: Vũ Thành Lộc

: Đặng Công Bách

: Nguyễn Quốc Dũng

: Đỗ Trọng Giáp

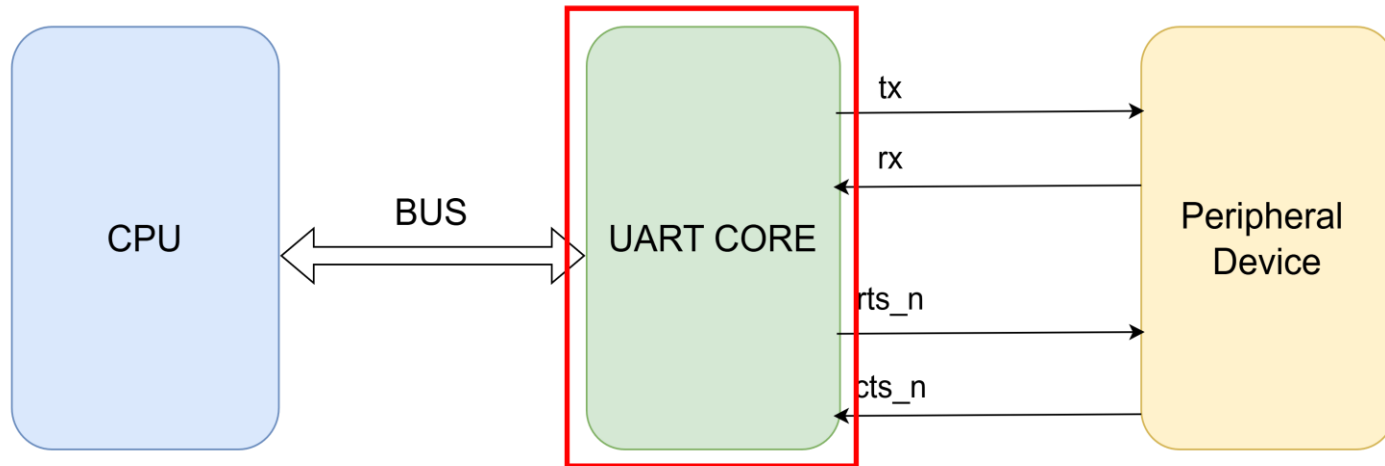
: Đỗ Thành Đạt

GVHD

: Nguyễn Đức Minh

- Tổng quan về UART
- Kiến trúc tổng thể
- Waveform
- Verify

1. Tổng quan về UART



- Hệ thống gồm 3 phần:
 - **CPU:** Đưa ra các lệnh điều khiển thông qua BUS để đưa đến UART CORE
 - **UART CORE:** Xử lí tín hiệu nhận vào và giao tiếp với CPU và ngoại vi
 - **Peripheral Device:** Ngoại vi

1. Tổng quan về UART

1. Cấu hình một frame truyền data

§ Cấu hình một frame truyền:

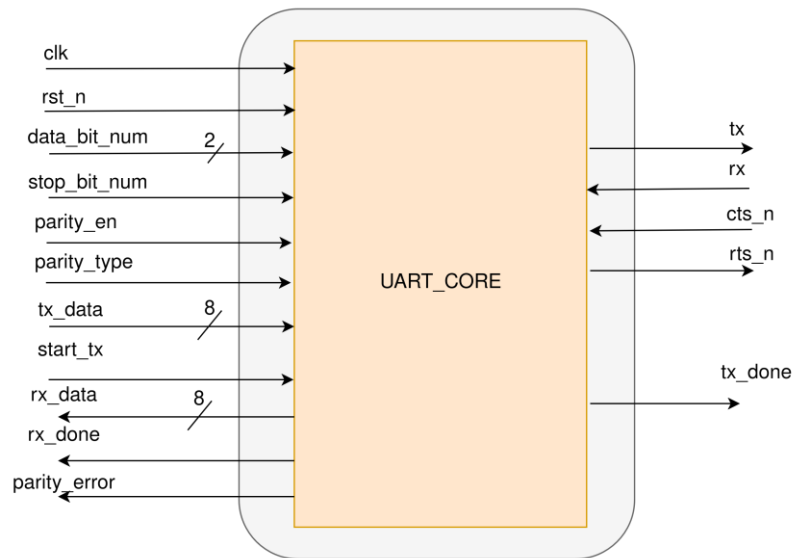
- **Bit data:** 5 - 8
- **Bit parity:** 0 hoặc 1
- **Parity type:** odd hoặc even
- **Bit stop:** 1 hoặc 2



Parity type:

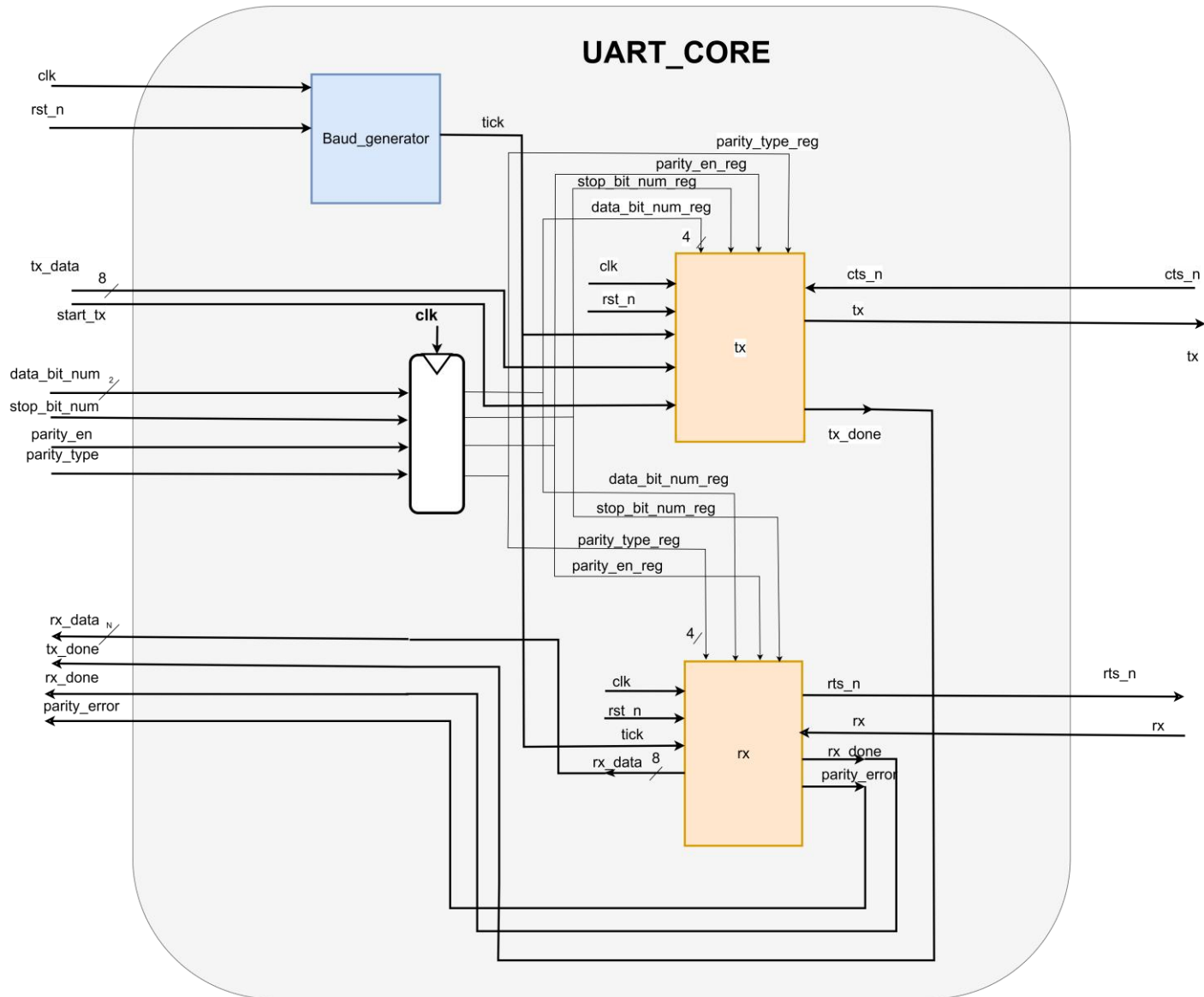
- **Odd (lẻ):** tổng số bit 1 trong toàn bộ frame (data + parity) truyền phải là lẻ
 - Ví dụ: data[7:0] = 8'b1011_0011 (lẻ bit 1) => parity bit = 0
 - data[6:0] = 7'b001_1000 (chẵn bit 1) => parity bit = 1
- **Even (chẵn) :** tổng số bit 1 trong toàn bộ frame truyền phải là chẵn
 - Ví dụ: data[7:0] = 8'b1011_0011 (lẻ bit 1) => parity bit = 1
 - data[6:0] = 7'b001_1000 (chẵn bit 1) => parity bit = 0

1. Kiến trúc tổng quan



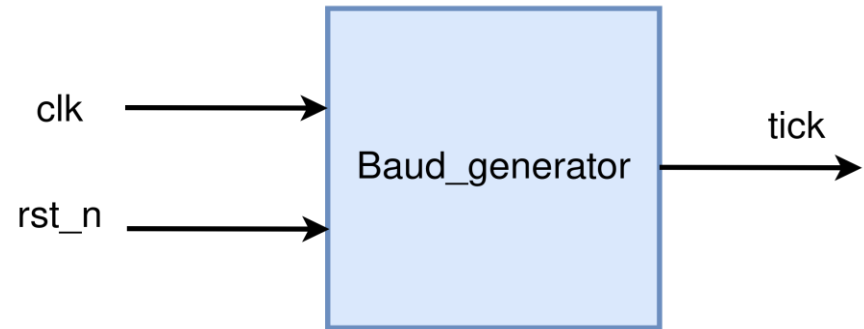
Signal	I/O	width	description
clk	input	1	Tín hiệu clock hệ thống
rst_n	input	1	Tín hiệu reset tích cực mức thấp
data_bit_num	input	2	Tín hiệu để giải mã ra số bit của data
stop_bit_num	input	1	Tín hiệu để giải mã ra số bit của stop
parity_en	input	1	Tín hiệu báo có parity bit hay không
parity_type	Input	1	Tín hiệu parity ODD/EVEN
tx_data	input	8	Data cần truyền ra chân tx
start_tx	input	1	Tín hiệu bắt đầu truyền data
tx_done	output	1	Tín hiệu báo đã truyền xong 1 transaction
rx_data	output	8	Data nhận được qua chân rx
rx_done	output	1	Tín hiệu báo rx đã nhận xong data
tx	output	1	Chân ra data nối tiếp
rx	input	1	Chân nhận data nối tiếp
rts_n	output	1	Tín hiệu bắt tay request-to-send
cts_n	input	1	Tín hiệu bắt tay clear-to-send

1. Kiến trúc tổng quan



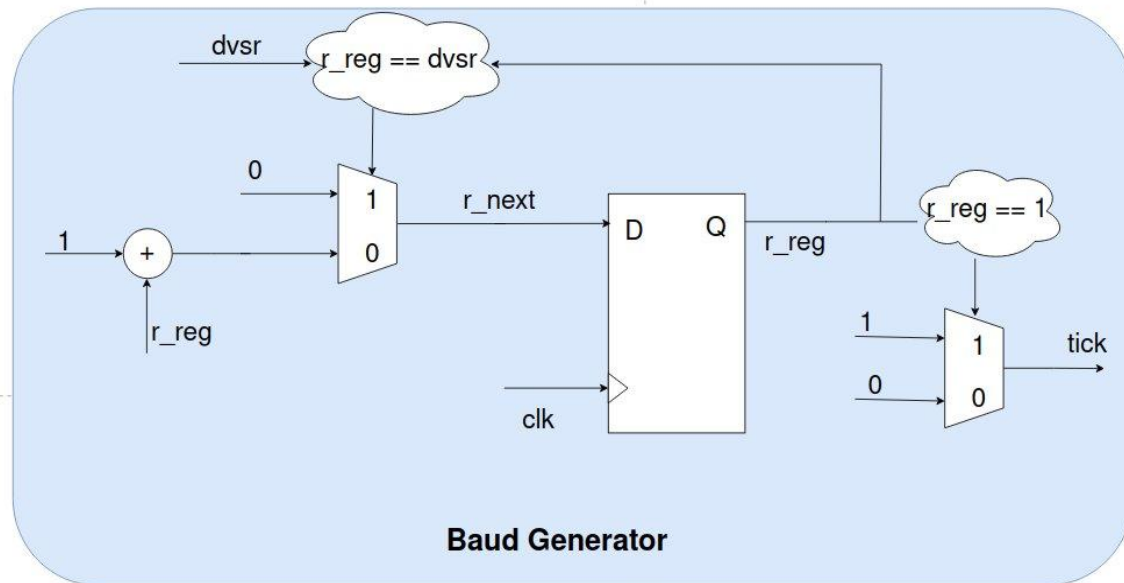
2.1. Baud Generator

- **Baud rate** là tốc độ truyền của UART đơn vị là bit/s
- **Baud_generator** là khối để sinh ra xung baud(tick) => Điều khiển TX, RX

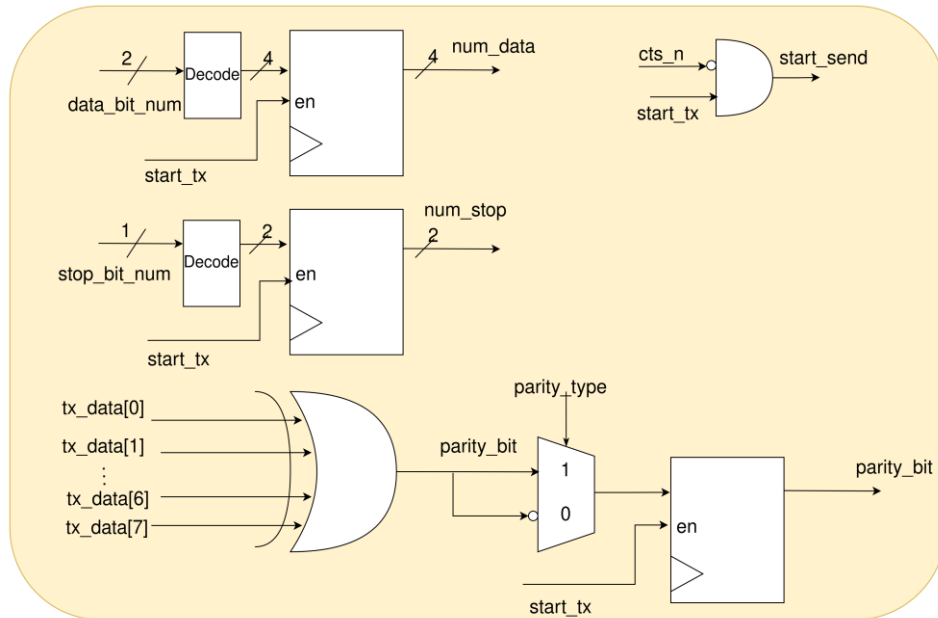
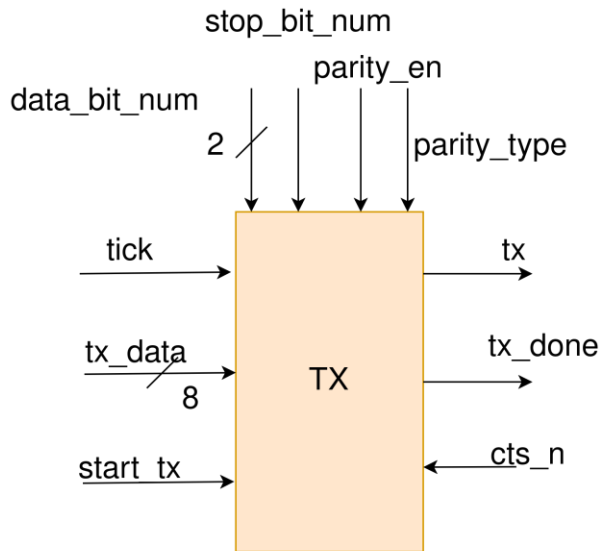


- **Công thức:**

$$dvsr = \frac{clk_hz}{baud\ rate \times 16}$$

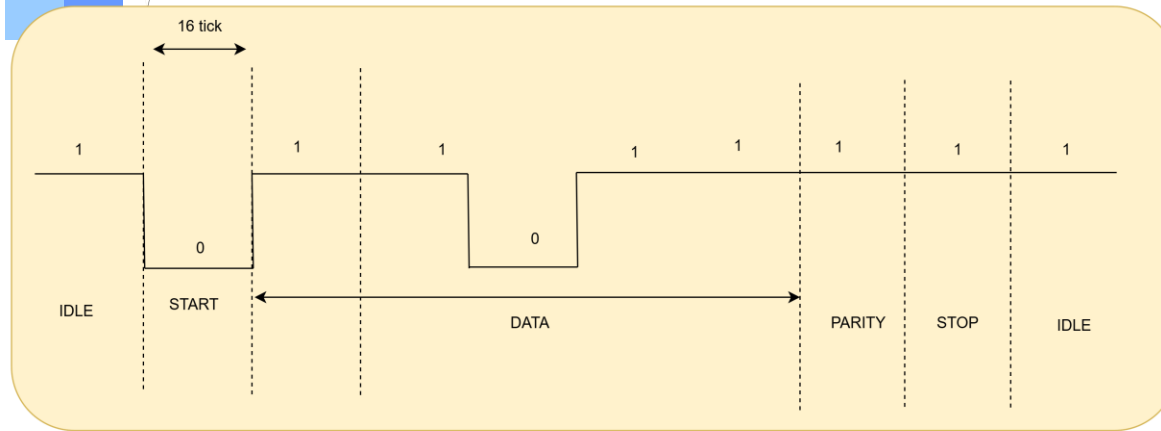


2.2. TX



Signal	I/O	width	description
clk	input	1	Tín hiệu clock hệ thống
rst_n	input	1	Tín hiệu reset tích cực mức thấp
tick	input	1	Tín hiệu từ baud generate ra để điều khiển
data_bit_num	input	2	Tín hiệu để giải mã ra số bit của data
stop_bit_num	input	1	Tín hiệu để giải mã ra số bit của stop
parity_en	input	1	Tín hiệu báo có parity bit hay không
tx_data	input	8	Data cần truyền ra chân tx
start_tx	input	1	Tín hiệu bắt đầu truyền data
tx_done	output	1	Tín hiệu báo đã truyền xong 1 transaction
tx	output	1	Chân ra data nối tiếp
cts_n	input	1	Tín hiệu bắt tay clear-to-send

- Chức năng:
 - Các tín hiệu đầu vào config và **tx_data** có nghĩa khi có **start_tx = 1** và **cts_n = 0** => bắt đầu truyền
 - Data sẽ được ra nối tiếp qua **tx**
 - Khi nào kết thúc truyền **tx_done = 1**

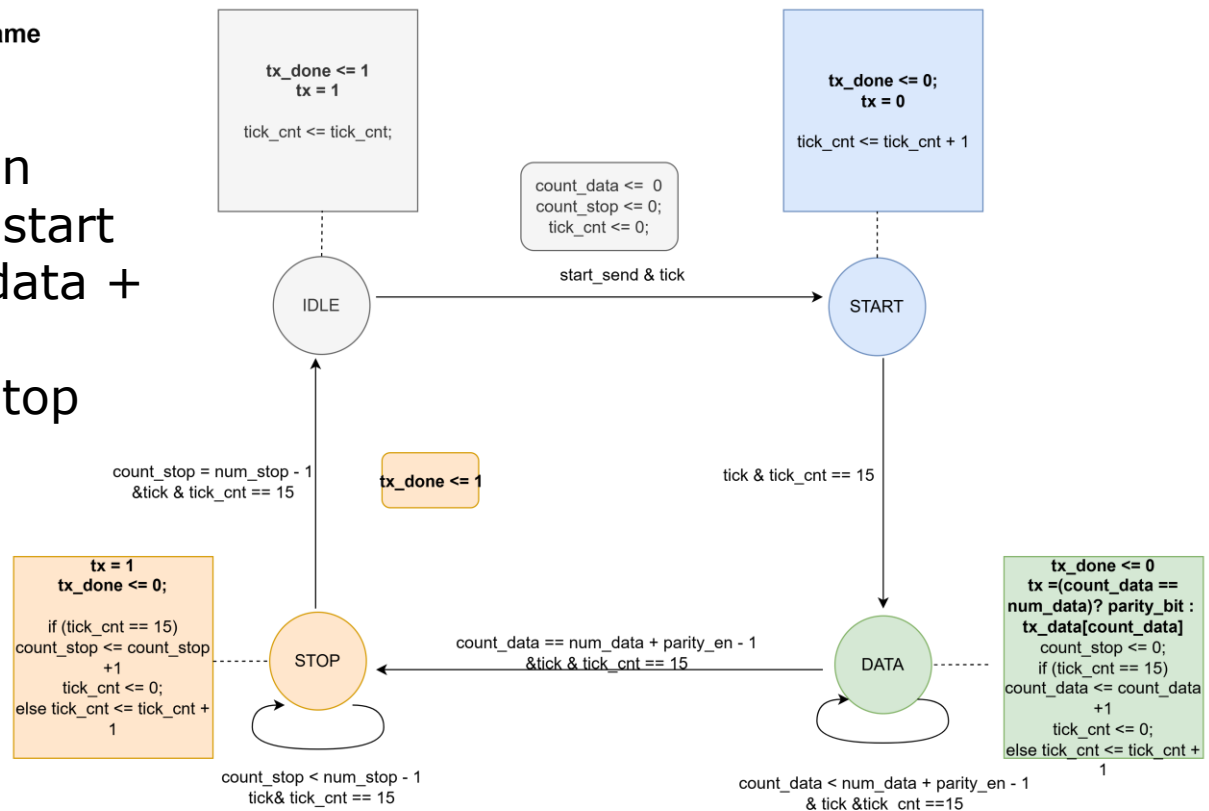


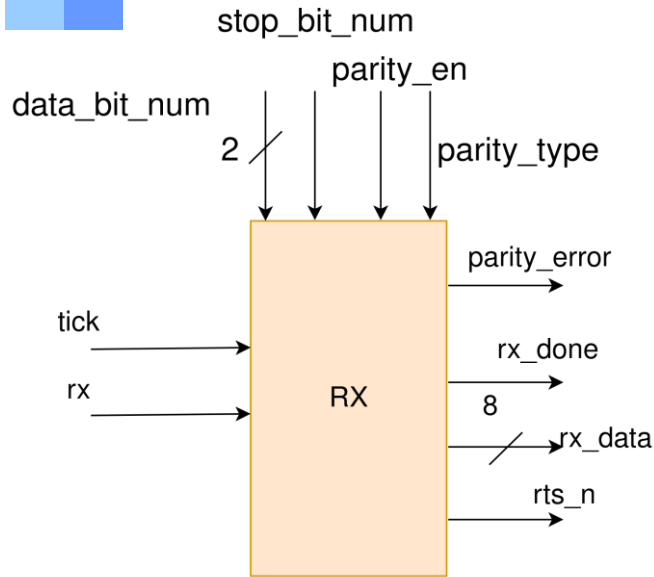
Data frame

- Ví dụ về data frame
 - 5 bit data
 - 1 bit parity
 - 1 bit stop

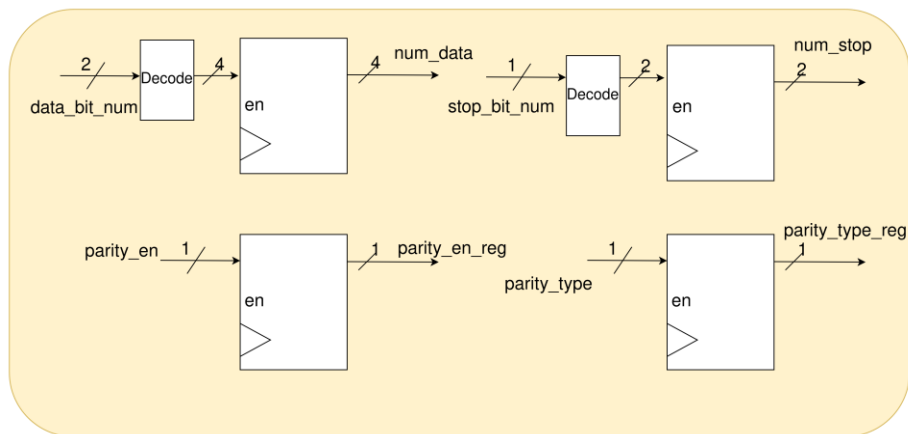
• 4 State chính

- **IDLE**: Không truyền
- **START**: truyền bit start
- **DATA**: Truyền bit data + parity
- **STOP**: Truyền bit stop





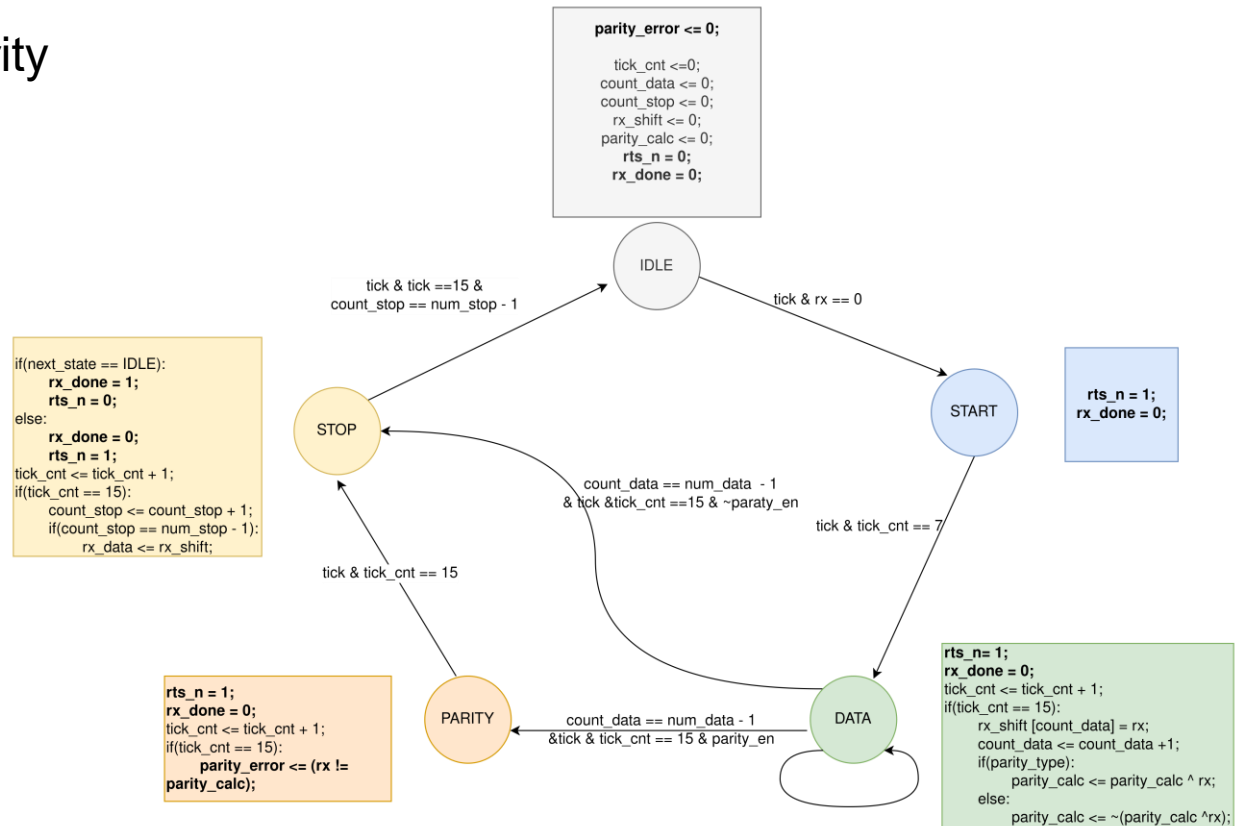
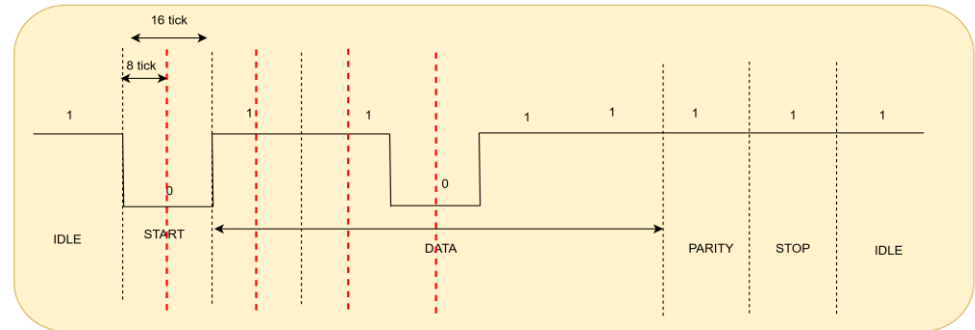
Signal	I/O	width	description
clk	input	1	Tín hiệu clock hệ thống
rst_n	input	1	Tín hiệu reset tích cực mức thấp
tick	input	1	Tín hiệu từ baud generate ra để điều khiển
data_bit_num	input	2	Số bit data
stop_bit_num	input	1	Số bit stop
parity_en	input	1	Tín hiệu báo có parity bit hay không
rx_data	output	8	Data nhận được qua chân rx
rx_done	output	1	Tín hiệu báo rx đã nhận xong data
rx	input	1	Chân nhận data nối tiếp
rts_n	output	1	Tín hiệu bắt tay request-to-send



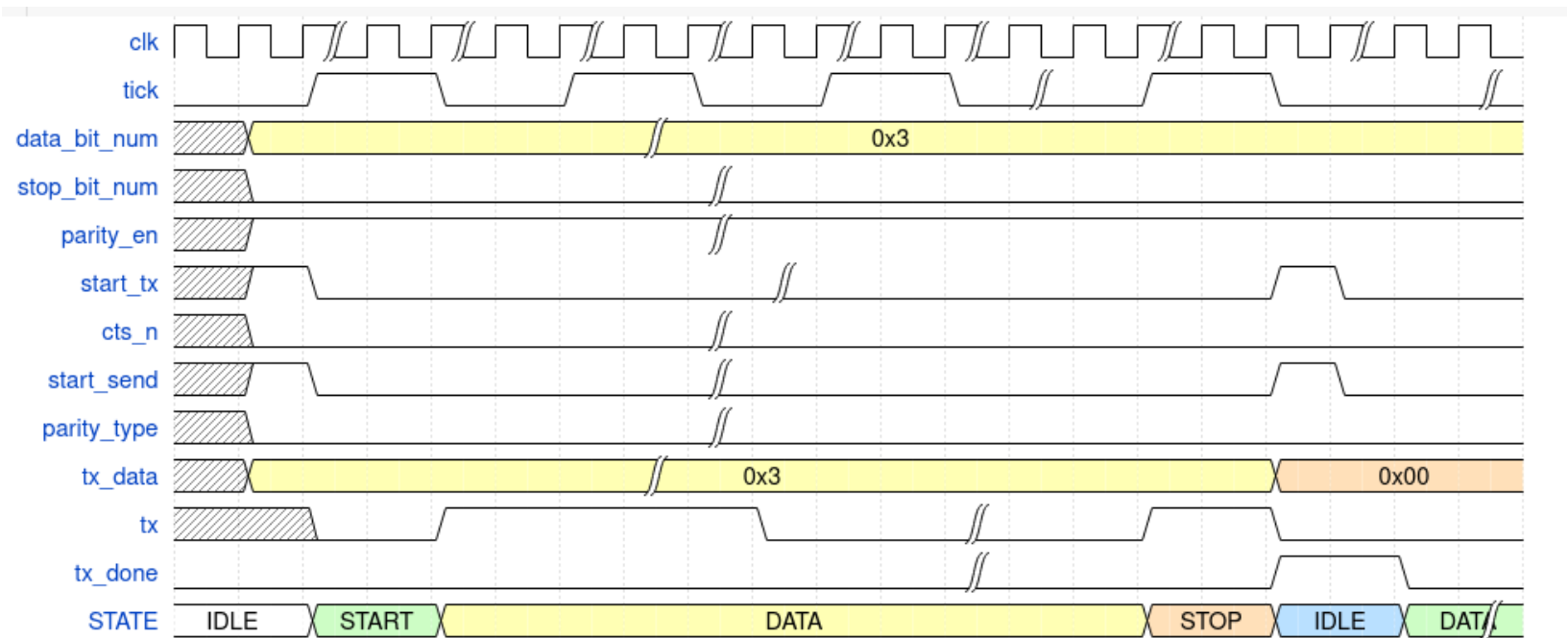
- Chức năng:
 - RX bắt đầu nhận data khi đang ở trạng thái **IDLE** và **rx = 0**
 - Các data từ chân **rx** nhận vào sẽ được lưu vào 1 thanh ghi **rx_data** ở bên trong
 - Khi nào RX nhận xong thì tín hiệu **rx_done = 1** và **rts_n = 0**
 - Chân **parity_error** sẽ kiểm tra bit parity

- 5 State:

- **IDLE**: Trạng thái nghỉ
- **START**: Trạng thái start
- **DATA**: Nhận bit data
- **PARITY**: Nhận bit parity
- **STOP**: Nhận bit stop

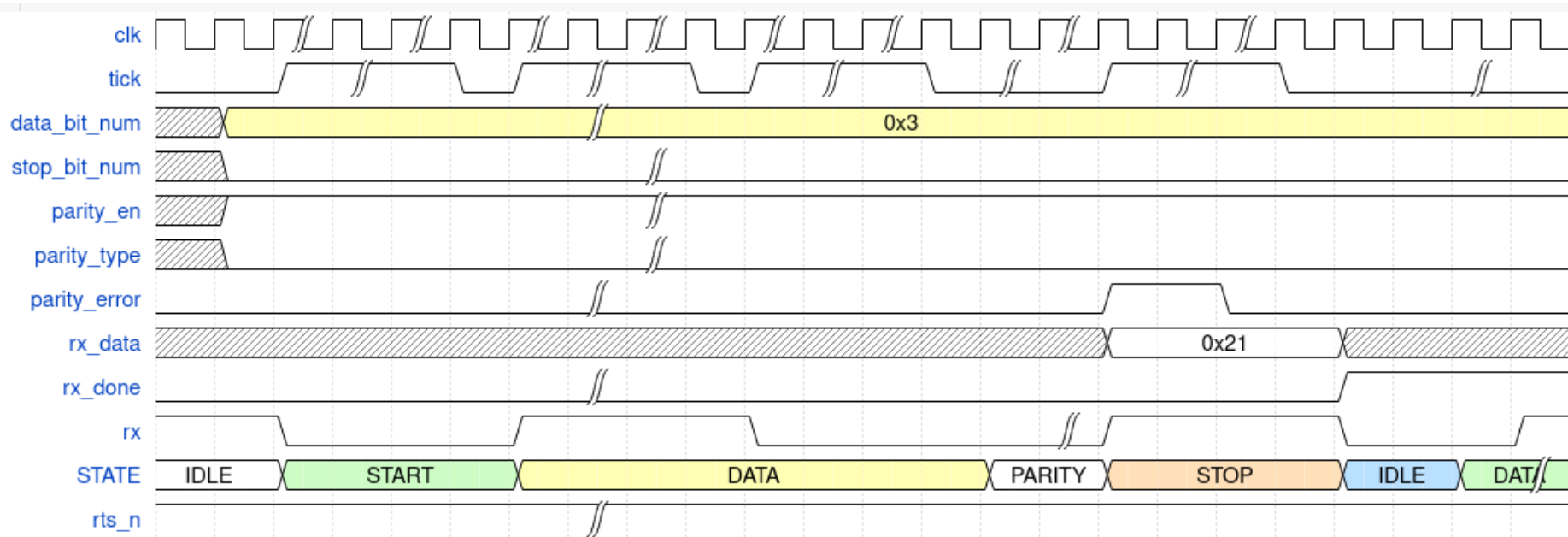


3. Waveform



Waveform tx

3. Waveform

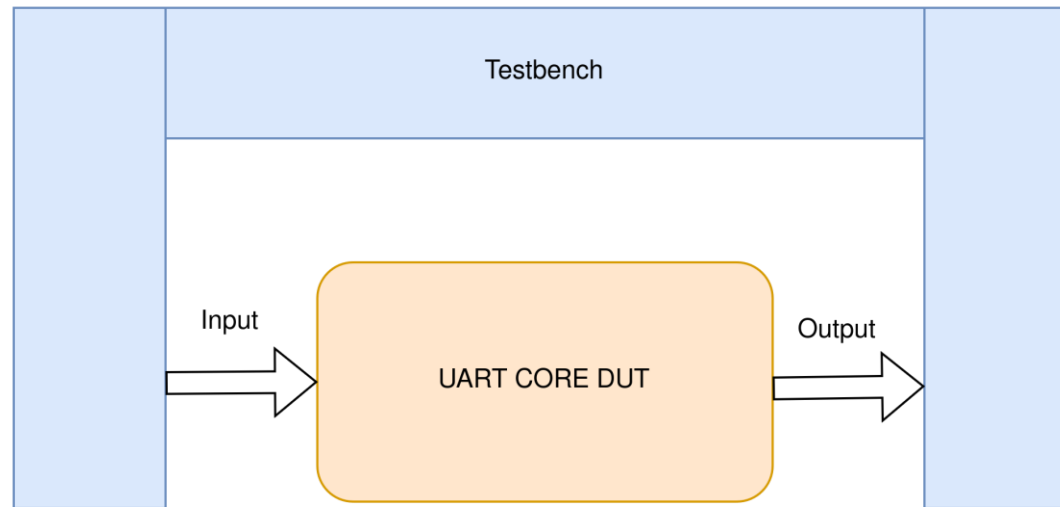


Waveform rx

4. Verify

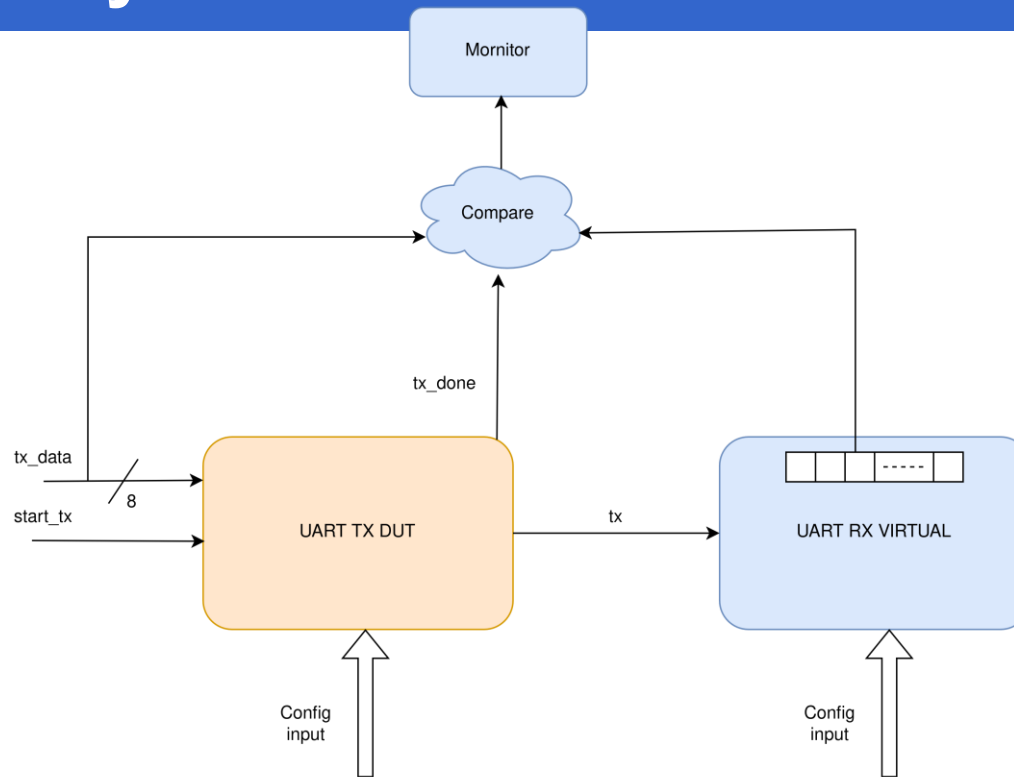
Function	Test case	Đầu vào	Số test	Kết quả
TX	Test parity bit	parity_en = 1 parity_type = odd, even	100	PASS
	Test các config kích thước frame	data_bit_num=2'b00 – 2'b11 Stop_bit_num = 1'b0 – 1'b1	800	PASS
	Thay đổi data, config trong quá trình truyền	Random	200	PASS
RX	Test parity error	parity_en = 1 parity_type = odd, even	100	PASS
	Test các config kích thước frame	data_bit_num=2'b00 – 2'b11 Stop_bit_num = 1'b0 – 1'b1	800	PASS
	Thay đổi data, config trong quá trình	Random	200	PASS

4. Verify



- Xây dựng testbench cho khối uart core
 - Đưa tín hiệu đầu vào trong uart core và quan sát đầu ra ở bên trong testbench
- Các chức năng chính cần kiểm tra
 - **Truyền data (TX)**
 - **Nhận data (RX)**

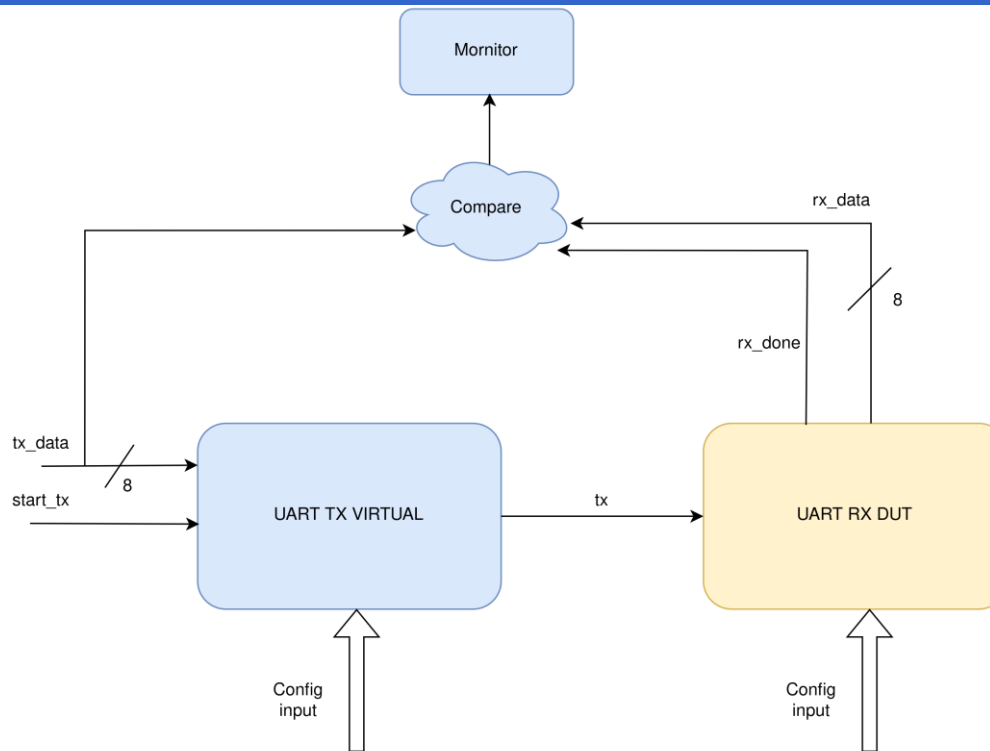
4. Verify



- Kiểm tra chức năng **TX**:

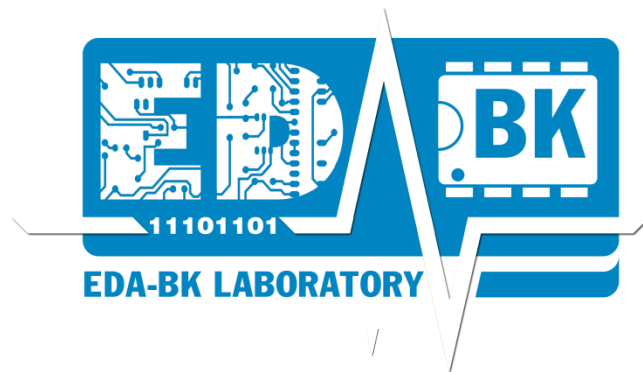
- Xây dựng **RX ảo** có chức năng được mô tả hành vi giống RX thật
- Data vào **TX dut** sẽ được truyền ra chân **tx** để đưa vào **RX**
- Sau khi truyền xong có tín hiệu **tx_done** => đưa vào khối **compare** để so sánh
- **Monitor**: In ra kết quả đúng hay sai

4. Verify



- Kiểm tra chức năng **RX**:
 - Xây dựng **TX ảo** có chức năng được mô tả hành vi giống **TX thật**
 - Data vào **TX** sẽ được truyền ra chân tx để đưa vào **RX dut**
 - Sau khi nhận xong có tín hiệu **rx_done** => đưa vào khối **compare** để so sánh
 - **Monitor**: In ra kết quả đúng hay sai

Thank You !



3. Testplan

1. Module RX

Test	Testcase
Test parity bit	Testcase1: 8bit data, odd parity, 1 bit stop Testcase2: 7bit data, even parity, 1bit stop Testcase3: 6bit data, odd parity, 1 bit stop Testcase4: 8bit data ,no parity bit, 1 bit stop Testcase 5: truyền sai parity bit (odd/even)
Test stop bit	Testcase1: 8bit data, không parity, 1 bit/ 2bit stop Testcase2: 8bit data, odd/even parity, 1 bit/ 2bit stop Testcase3: không có stop bit
Test data	Testcase 1: 8 bit data, odd/even parity bit, 1bit/2bit stop Testcase 2: 7 bit data, odd/even parity bit, 1bit/2bit stop Testcase 3: 6 bit data, odd/even parity bit, 1bit/2bit stop Testcase 4: 5 bit data, odd/even parity bit, 1bit/2bit stop
Random test	Testcase 1: random, data_bit_num, parity_en, parity_type, stop bit num